

SPACEBATTLE

고전 슈팅 게임

2026. 02. 27

게임플레이 & 주요 특징

◎ 게임 구조

플레이어는 기체를 조종하여 끊임없이 몰려오는 적군을 물리치고, 스코어가 300점이 될 때 마다 생성되는 보스를 처치하면 난이도가 상승되어 가능한 한 오랫동안 생존하여 **최고 점수(High Score)**를 획득해야 합니다.



직관적 조작

방향키와 스페이스바만을 이용하여 단순하고 명확한 조작 체계



생존 시스템

보스 처치시 추가되는 라이프 시스템



전투

다양한 위치에서 생성되는 적군을 제거



MOVE

적의 공격 회피



SHOOT

미사일 발사 및 격추



SCORE

점수 획득 및 랭킹

플레이어 및 객체



#player

```
const player = {  
  x: canvas.width / 2 - 30, // 화면 중앙 위치  
  y: canvas.height - 80, // 화면 아래 위치  
  width: 60,  
  height: 60,  
  speed: 5 // 이동 속도  
};
```

#enemy

```
let enemies = [];  
let enemyCount = 3;  
  
function createEnemy() {  
  return {  
    x: Math.random() * (canvas.width - 60),  
    y: 0,  
    width: 60,  
    height: 60,  
    speed: 2 + level * 0.3 // 레벨이 오르면 빨라짐  
  };  
}
```

#boss

```
function createBoss() {  
  return {  
    x: canvas.width / 2 - 100,  
    y: 60,  
    width: 200,  
    height: 120,  
    hp: 100 + level * 20,  
    maxHp: 100 + level * 20,  
    speed: 3 + level * 0.3,  
    dir: 1  
  };  
}
```

player , enemy , boss

플레이어 , 적 , 보스를 묶어서 생성하는것이 아닌
3가지의 객체로 만들어서 유지 보수가 쉽게
확장을 구조화 하였습니다.

화면처리

#시작 화면

```
if (!gameStarted) {  
  ctx.textAlign = "center";  
  ctx.fillStyle = "white";  
  ctx.font = "bold 45px Arial";  
  ctx.fillText("🚀 SPACE BATTLE 🚀",  
    canvas.width / 2, canvas.height / 2);  
  requestAnimationFrame(gameLoop);  
  return;  
}
```

게임 오버 화면

```
if (gameOver) {  
  ctx.textAlign = "center";  
  ctx.fillStyle = "white";  
  ctx.font = "bold 45px Arial";  
  ctx.fillText("GAME OVER", canvas.width / 2, canvas.height / 2);  
  requestAnimationFrame(gameLoop);  
  return;  
}
```

#setInterval

`setInterval` 사용시 실행기준이 시간(ms) 기준으로 실행이 되며 탭이 비활성화 되어있을 때도 계속 실행이 유지됩니다. 그로인하여 프레임이 끊기는 현상이 발생할 수 있습니다.

#requestAnimationFrame

`requestAnimationFrame`은 실행기준이 브라우저에서 1초에 60번씩 화면을 구현하기 때문에 화면이 그려질때 맞춰서 실행됩니다. 탭이 비활성화 되어있을때 자동으로 멈추며 `setInterval`과 다르게 프레임 드랍없이 깔끔하게 구현가능

보스 등장 시 Warning 문구 출력



#보스 등장 시 Warning 처리

1. **warningActive** 상태 시
2. 게임 로직을 멈춤
3. 타이머 감소
4. 시간이 끝나면 보스 생성
5. 그동안 다른 로직은 실행되지 않음

게임을 진행 중 특정 점수가 되었을 때 자동으로 보스가 생성
 게임 중 보스가 갑자기 생성되면 게임의 흐름이
 자연스럽게 안기 때문에 보스 등장 전 **Warning** 문구를 추가
 게임의 흐름이 끊어지지 않게끔 하기 위하여 구현

상태 기반 구조 기반 설계

#상태 기반 구조

if(gamestarted) : 게임시작 시

if(warningActive) : 보스 출현 전 워닝문구 출력 시

if(bossActive) : 보스 출현 중

if(gameOver) : 게임패배 시

1. 시작 상태 (gameStarted = false)
2. 일반 전투 상태
3. WARNING 상태 (warningActive = true)
4. 보스 전투 상태 (bossActive = true)
5. 게임 오버 상태 (gameOver = true)

이 게임은 상태 기반 구조로 설계되었습니다.
시작 화면, 일반 전투, 보스 경고 상태, 보스전, 게임오버 상태를
각각 분리하여 현재 상태에 따라 다른 로직이 실행되도록 구성했습니다.
이를 통해 코드가 명확해지고 확장성이 높아졌습니다.

충돌 및 무적 처리

#충돌처리

```
function isColliding(a, b) {
```

```
  // 원형 충돌 (총알)
```

```
  if (a.radius) {
```

```
    return (
```

```
      a.x + a.radius > b.x &&
```

```
      a.x - a.radius < b.x + b.width
```

```
    &&
```

```
      a.y + a.radius > b.y &&
```

```
      a.y - a.radius < b.y + b.height
```

```
    );
```

```
  }
```

```
  // 사각형 충돌
```

```
  return (
```

```
    a.x < b.x + b.width &&
```

```
    a.x + a.width > b.x &&
```

```
    a.y < b.y + b.height &&
```

```
    a.y + a.height > b.y
```

```
  );
```

```
}
```

#무적처리

```
let invincible = false;
```

```
if (isColliding(player, boss)
```

```
&& !invincible) {
```

```
  lives--;
```

```
  invincible = true;
```

```
  setTimeout(() => invincible
```

```
= false, 1000);
```

```
}
```

#충돌처리

총알은 원형, 캐릭터는 사각형이기 때문에 하나의 함수에서 두 가지 충돌을 모두 처리하도록 했습니다.
이를 통해 코드 중복을 줄이고 재사용성을 높였습니다.

#무적처리

연속적인 적이 한꺼번에 부딪히는 경우를 대비 하여
전역 변수를 생성 하여 목숨이 낮아 졌을 때 활성화 시켜
1초간 무적이 되는 기능을 추가

느낀점

느낀점

이번 프로젝트를 진행하면서 단순히 게임을 구현하는 것이 아니라, 전체 구조를 설계하는 과정이 얼마나 중요한지 느끼게 되었습니다. 처음에는 캐릭터를 움직이고 공격 기능만 구현하면 된다고 생각했지만, 실제로 구현해보니 게임의 흐름을 어떻게 나눌지, 상태를 어떻게 관리할지 고민하는 시간이 더 많았습니다.

특히 상태 기반 구조를 적용하면서 시작 화면, 전투 상태, 보스 등장 전 WARNING 상태, 보스전, 게임 오버 상태를 명확히 분리하는 것이 중요하다는 것을 배웠습니다.

또한 충돌 처리 과정에서 연속으로 목숨이 줄어드는 문제가 발생했는데, 이를 해결하기 위해 무적 타이머 시스템을 도입하면서 문제를 분석하고 해결하는 경험을 할 수 있었습니다.

이번 프로젝트를 통해 단순 기능 구현을 넘어 구조 설계와 확장성을 고려하는 사고 방식이 중요하다는 것을 깨닫게 되었고, 이후에는 보스 패턴이나 아이템 시스템 등 더 다양한 기능을 추가해보고 싶다는 생각이 들었습니다.